

Untitled page

JDBC (Java Database Connectivity) — Complete Study Notes

1. Introduction to JDBC

JDBC = Java Database Connectivity

What is JDBC?

JDBC is the bridge that lets a Java program talk to a database. Without it, Java would only know how to work with **files** (via `java.io`), not with structured, queryable data stored in a database.

Data → Information

Java

```
├── Hard Disk (file)    → handled using java.io (file handling)
└── Database           → handled using JDBC
```

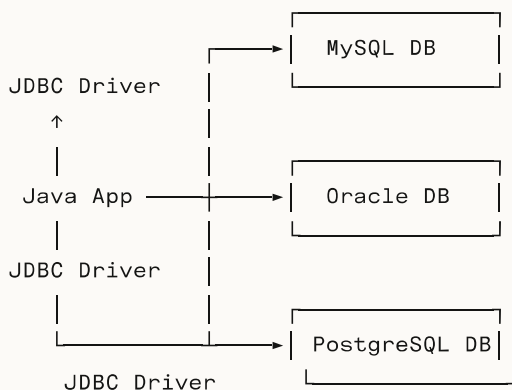
Why do we need JDBC?

- Files are unstructured and hard to query, filter, or relate.
- Databases store data in **tables** and allow powerful querying (SQL).
- Java needs a **standard way** to connect to *any* database (MySQL, Oracle, PostgreSQL, etc.) — that standard is JDBC.

Architecture Overview

```
Java App/Program <-----> JDBC <-----> Database (Table)
                                     (local / server / remote / cloud)
```

A single Java application can talk to **different databases** using **different JDBC drivers** — the driver is what translates JDBC calls into database-specific instructions.



Each database vendor (MySQL, Oracle, PostgreSQL...) provides its **own JDBC driver** (a JAR file) that must be added to the project.

File Handling vs. Database Connectivity

Aspect	File Handling (<code>java.io</code>)	Database Connectivity (JDBC)
Storage	Hard disk file	Database (table)
Structure	Unstructured / flat	Structured (rows & columns)
Package	<code>java.io.*</code>	<code>java.sql.*</code>
Query capability	Manual parsing	SQL queries
Location	Local only	Local / Server / Remote / Cloud

2. Steps Involved in Developing a JDBC Application

There are **7 core steps** to any JDBC program:

1. **Import the required package** (also download and add the database-specific JAR to the project)
2. **Load and Register the Driver**
3. **Establish the Connection**
4. **Create the Statement**
5. **Execute the Query**
6. **Process the Result**
7. **Close the resources**

Steps involved in developing JDBC Application

- Import the required package
(Also download & add Database specific jar into the project)
- Load and Register the Driver
- Establish the Connection
- Create the statement
- Execute the Query
- Process the Result
- Close

3. Database Setup

Before writing Java code, the database and table must exist.

Create database and table (MySQL Workbench):

```
use jdbclearning;

CREATE table studentInfo (
    id      INT not null,
    sname   VARCHAR(40) not null,
    sage    INT not null,
    scity   VARCHAR(40) not null,
    PRIMARY KEY(id)
```

```
);
```

Table structure — `studentInfo`:

Column	Data Type	Constraint
id	INT	NOT NULL, PRIMARY KEY
sname	VARCHAR(40)	NOT NULL
sage	INT	NOT NULL
scity	VARCHAR(40)	NOT NULL

4. Driver Loading & Registration

`Class.forName()`

Used to **load and register** the JDBC driver class into memory *before* a connection is attempted.

```
//Load and Register the Driver
Class.forName("com.mysql.cj.jdbc.Driver");
```

Why is it used?

`Class.forName("...")` loads a class **by name at runtime** and triggers its **static block** to execute. JDBC drivers use a static block internally to register themselves with `DriverManager`. So calling `Class.forName()` on the driver class ensures the driver is registered and ready to be used — even though you never explicitly create an object of it.

Demonstration — how `Class.forName()` triggers a static block

```
public class LaunchClassForNameEx {
    public static void main(String[] args) throws ClassNotFoundException {
        Class.forName("Demo");
    }
}

class Demo {
    static {
        System.out.println("Static Block");
    }
}
```

Console Output:

```
Static Block
```

This proves `Class.forName()` loads the class into the JVM and runs its static initializer — exactly what a JDBC driver relies on to self-register with `DriverManager`.

5. Establishing the Connection

Connection URL Format

```
jdbc:<databaseType>://<host>:<port>/<databaseName>
```

Example:

```
jdbc:mysql://localhost:3306/jdbclearning
```

Code

```
//Establish the connection
String url = "jdbc:mysql://localhost:3306/jdbclearning";
String user = "root";
String password = "root123";
Connection connect = DriverManager.getConnection(url, user, password);
```

- **DriverManager** — a class in `java.sql` that manages the set of registered JDBC drivers and hands out `Connection` objects.
- **Connection** — an interface representing an active session with the database.

6. Statement & Query Execution

Creating a Statement

```
//creating statement
Statement statement = connect.createStatement();
```

`Statement` is used to send SQL queries to the database and get results back.

a) INSERT Operation

```
package com.telusko.jdbclearning;
import java.sql.*;

public class LaunchApp1 {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //Load and Register the Driver
        Class.forName("com.mysql.cj.jdbc.Driver");

        //Establish the connection
        String url = "jdbc:mysql://localhost:3306/jdbclearning";
        String user = "root";
        String password = "root123";
        Connection connect = DriverManager.getConnection(url, user, password);

        //creating statement
        Statement statement = connect.createStatement();

        //execute query
```

```
String sql = "INSERT INTO studentinfo(id, sname, sage, scity) VALUES(1, 'Rohan', 17, 'Bengaluru')";
int rowAffected = statement.executeUpdate(sql);

//process the result
if (rowAffected == 0) {
    System.out.println("Unable to insert the data");
} else {
    System.out.println("Data Inserted Successfully!");
}

//close the resources
statement.close();
connect.close();
}
```

- `executeUpdate(sql)` is used for **DML operations** (INSERT, UPDATE, DELETE) — it returns the **number of rows affected** as an `int`.
- If `rowAffected == 0` → operation had no effect (failure/no match).

Verify in MySQL Workbench:

```
select * from studentinfo;
```

Result:

id	sname	sage	scity
1	Rohan	17	Bengaluru

b) UPDATE Operation

```
//execute query
String sql = "UPDATE studentinfo set sage=24 where id=2";
int rowAffected = statement.executeUpdate(sql);
if (rowAffected == 0)
    System.out.println("Updation failed");
else
    System.out.println("Update successfull!");

//close the resources
statement.close();
connect.close();
```

c) SELECT Operation

```
//execute query
String sql = "SELECT * FROM studentinfo";
ResultSet rs = statement.executeQuery(sql);
```

```
//process the result
while (rs.next()) {
    System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getInt(3) + " " +
rs.getString(4));
}

//close the resources
rs.close();
statement.close();
connect.close();
```

- `executeQuery(sql)` is used for **SELECT** statements — it returns a `ResultSet`.
- `ResultSet` is like a cursor pointing to rows of data. `rs.next()` moves the cursor to the next row and returns `false` when there are no more rows.
- Column values can be accessed by **index** (`getInt(1)`) or by **column name** (`getInt("id")`).

Better readability — using column names instead of indices:

```
System.out.println(
    rs.getInt("id") + " " + rs.getString("sname") + " " +
    rs.getInt("sage") + " " + rs.getString("scity")
);
```

Console Output:

```
1 Rohan 17 Bengaluru
2 Rohit 24 Bengaluru
```

Access method	Example	Notes
By column index	<code>rs.getInt(1)</code>	Faster, but fragile if column order changes
By column name	<code>rs.getInt("id")</code>	More readable & robust

d) DELETE Operation

```
//execute query
String sql = "DELETE FROM studentinfo where id=2";
int rowsAffected = statement.executeUpdate(sql);

//process the result
if (rowsAffected == 0)
    System.out.println("Failed to delete the record");
else
    System.out.println("Record deleted Successfully!");
```

Console Output:

Record deleted Successfully!

7. Advanced Execution: `execute()` Method

Why `execute()`?

- `executeQuery()` → only for SELECT (returns `ResultSet`)
- `executeUpdate()` → only for INSERT/UPDATE/DELETE (returns `int` rows affected)
- `execute()` → works for **any** SQL statement (SELECT or DML) and returns a `boolean`:
 - `true` → the result is a `ResultSet` (i.e., it was a SELECT)
 - `false` → the result is an update count (i.e., it was INSERT/UPDATE/DELETE)

Retrieving the actual result after `execute()`

If <code>execute()</code> returns	Use this to get the result
<code>true</code> (SELECT)	<code>statement.getResultSet()</code>
<code>false</code> (INSERT/UPDATE/DELETE)	<code>statement.getUpdateCount()</code>

Full Example

```
String sql = "SELECT * FROM studentinfo";
//execute query
boolean status = statement.execute(sql);

//Process the result
if (status) {
    //select
    ResultSet rs = statement.getResultSet();
    while (rs.next()) {
        System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getInt(3) + " " +
rs.getString(4));
    }
} else {
    //insert, update, delete
    int rows = statement.getUpdateCount();
    if (rows == 0)
        System.out.println("Operation failed!");
    else
        System.out.println("Operation successfull!");
}
```

Same logic tested with an UPDATE query

```
String sql = "Update studentinfo set sage=16 where id=1";
```

```
//execute query
boolean status = statement.execute(sql);
//Process the result
if (status) {
    System.out.println("If block");
    //select ...
} else {
    // update, delete
    int rows = statement.getUpdateCount();
    if (rows == 0)
        System.out.println("Operation failed!");
    else
        System.out.println("Operation successfull!");
}
```

Console Output:

```
else block
Operation successfull!
```

This confirms: for an UPDATE query, `execute()` returns `false`, so the `else` branch runs and `getUpdateCount()` reports the affected rows.

8. Exception Handling in JDBC

JDBC operations can throw checked exceptions (`ClassNotFoundException`, `SQLException`), so production code wraps them in **try-catch-finally**.

```
Connection connect = null;
Statement statement = null;

try {
    //Load and Register the Driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    //Establish the connection
    String url = "jdbc:mysql://localhost:3306/jdbclearning";
    String user = "root";
    String password = "root123";
    connect = DriverManager.getConnection(url, user, password);

    //creating statement
    statement = connect.createStatement();

    // ... execute query & process result ...

}
catch (ClassNotFoundException e) {
    e.printStackTrace();
}
catch (SQLException e) {
    e.printStackTrace();
}
```



```

catch (Exception e) {
    e.printStackTrace();
}
finally {
    //close the resources
    try {
        statement.close();
        connect.close();
    }
    catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

Key points

- `Connection` and `Statement` are declared **outside** the try block (initialized to `null`) so they remain accessible in the `finally` block.
- Multiple `catch` blocks handle different exception types: `ClassNotFoundException` (driver not found), `SQLException` (database errors), `Exception` (catch-all).
- The `finally` block **always executes**, guaranteeing resources are closed — even if an exception occurs. Since `close()` itself can throw `SQLException`, it needs its own nested try-catch.

9. JDBC Utility Class (`JdbcUtil`)

To avoid repeating connection/closing boilerplate in every class, extract it into a reusable utility class.

```

public class JdbcUtil {

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        }
        catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }

    public static Connection getConnection() throws SQLException {
        //Establish the connection
        String url = "jdbc:mysql://localhost:3306/jdbclearning";
        String user = "root";
        String password = "root123";
        return DriverManager.getConnection(url, user, password);
    }

    public static void closeConnection(Connection connect, Statement statement) throws
    SQLException {
        statement.close();
        connect.close();
    }
}

```

Using `JdbcUtil` in a program

```
try {
    connect = JdbcUtil.getConnection();

    //creating statement
    statement = connect.createStatement();

    //execute query
    String sql = "INSERT INTO studentinfo(id, sname, sage, scity) VALUES(3, 'Rohit', ...)";
    int rowAffected = statement.executeUpdate(sql);

    //process the result
    if (rowAffected == 0)
        System.out.println("Unable to insert the data");
    else
        System.out.println("Data Inserted Successfully!");
}
catch (SQLException e) {
    e.printStackTrace();
}
catch (Exception e) {
    e.printStackTrace();
}
```

Benefits of a utility class

- **DRY code** — driver loading, connection setup, and closing logic written once.
- **Consistency** — every class connects the same way.
- **Easier maintenance** — change the URL/credentials in one place only.

10. Resource Management — Best Practices

Always close, in reverse order of creation

```
Open order:    Connection → Statement → ResultSet
Close order:   ResultSet  → Statement → Connection
```

```
rs.close();
statement.close();
connect.close();
```

Best Practices

- Always close `ResultSet`, `Statement`, and `Connection` — leaked connections exhaust the database's connection pool.
- Close resources inside a `finally` block (or use try-with-resources) so they close even if an exception occurs.
- Use a **utility class** to centralize connection creation/closing logic.
- Use **column names** instead of indices in `ResultSet.get...()` calls for readability and resilience to schema changes.

- Use `executeUpdate()` for DML, `executeQuery()` for SELECT, and `execute()` only when the query type isn't known in advance.
- Never hardcode credentials in multiple places — centralize them (e.g., in `JdbcUtil`).

Common Pitfalls

Pitfall	Consequence	Fix
Forgetting to close <code>Connection</code> / <code>Statement</code>	Connection leaks, DB runs out of connections	Close in <code>finally</code>
Using <code>executeQuery()</code> for INSERT/UPDATE/DELETE	Throws <code>SQLException</code>	Use <code>executeUpdate()</code>
Using <code>executeUpdate()</code> for SELECT	Throws <code>SQLException</code>	Use <code>executeQuery()</code>
Forgetting <code>Class.forName()</code> (older JDBC versions)	<code>Driver not found</code> type errors	Always load driver first
Hardcoding SQL with string concatenation of user input	SQL Injection risk	Prefer <code>PreparedStatement</code> (not covered in these notes but the natural next step)

11. Project Structure Overview

```

FirstJdbcApp
├── src
│   ├── (default package)
│   │   └── LaunchClassForNameEx.java    → demonstrates Class.forName() + static block
│   └── com.telusko.jdbclearning
│       ├── JdbcUtil.java                → reusable getConnection() & closeConnection()
│       ├── LaunchApp1.java              → INSERT operation
│       ├── LaunchApp2.java              → UPDATE operation
│       ├── LaunchApp3.java              → SELECT operation (index-based, then name-based)
│       ├── LaunchApp4.java              → DELETE operation
│       ├── LaunchApp5.java              → execute() with boolean status handling
│       └── LaunchApp6.java              → Full try-catch-finally + JdbcUtil integration

```

Class Responsibilities Summary

Class	Purpose	Key Method Used
<code>LaunchClassForNameEx</code>	Demonstrates driver-loading mechanism	<code>Class.forName()</code>

LaunchApp1	Insert a new student record	executeUpdate()
LaunchApp2	Update an existing student's age	executeUpdate()
LaunchApp3	Fetch and print all students	executeQuery() + ResultSet
LaunchApp4	Delete a student record	executeUpdate()
LaunchApp5	Generic execution handling SELECT/DML	execute(), getResultSet(), getUpdateCount()
LaunchApp6	Production-style exception handling	try-catch-finally
JdbcUtil	Centralized connection management	getConnection(), closeConnection())

12. Quick Reference — Method Cheat Sheet

Method	Used For	Returns
Class.forName(driverClass)	Load & register JDBC driver	void (throws ClassNotFoundException)
DriverManager.getConnection(url, user, pass)	Open a DB connection	Connection
connection.createStatement()	Create a Statement object	Statement
statement.executeUpdate(sql)	Run INSERT / UPDATE / DELETE	int (rows affected)
statement.executeQuery(sql)	Run SELECT	ResultSet
statement.execute(sql)	Run any SQL (unknown type)	boolean (true = SELECT)
statement.getResultSet()	Get result after execute() returns true	ResultSet
statement.getUpdateCount()	Get affected rows after execute() returns false	int
resultSet.next()	Move cursor to next row	boolean
resultSet.getInt(...)/ getString(...)	Read column value (by index or name)	int / String
.close() (on ResultSet, Statement, Connection)	Release resource	void

13. End-to-End Flow Diagram

1. Import package + add DB-specific JAR (e.g., mysql-connector)	
2. <code>Class.forName("com.mysql.cj.jdbc.Driver")</code>	— loads driver
3. <code>DriverManager.getConnection(url,user,pass)</code>	— opens Connection
4. <code>connection.createStatement()</code>	— creates Statement
5. <code>statement.execute*(sql)</code>	— runs SQL
6. Process result (ResultSet / rows affected)	
7. Close ResultSet → Statement → Connection	

Notes compiled from JDBC lecture screenshots (Telusko series) covering: JDBC introduction, the 7-step workflow, database/table setup, driver loading, connection establishment, CRUD operations (Insert/Select/Update/Delete), the generic `execute()` method, exception handling, and building a reusable `JdbcUtil` class.